COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	10/1/26	**DATE SUBMITTED:**	10/1/26
NAME:	Medina, Jomz Ryle E.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 10.1 **Graphs**

1. PROCEDURE OUTPUT

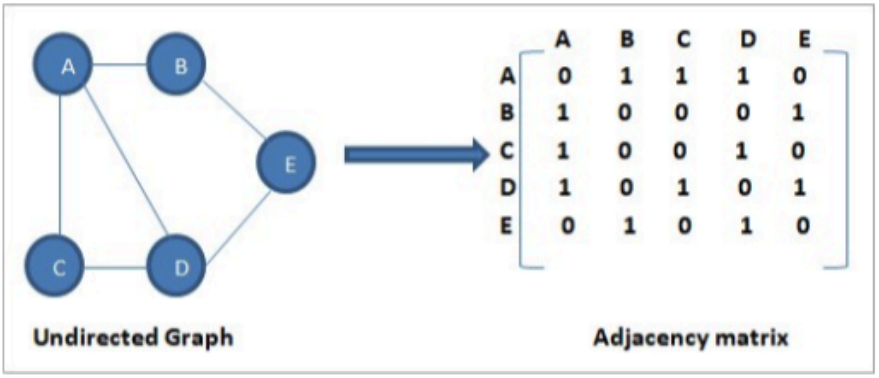
ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

A.1. Create a Graph

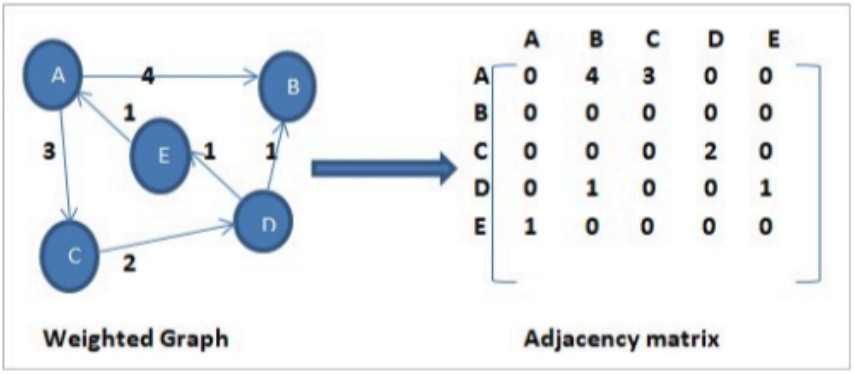
Following are the basic operations that we can perform on the graph data structure:

- **Add a vertex:** Adds vertex to the graph.
- **Add an edge:** Adds an edge between the two vertices of a graph.
- **Display the graph vertices:** Display the vertices of a graph.

A.2. Adjacency Matrix

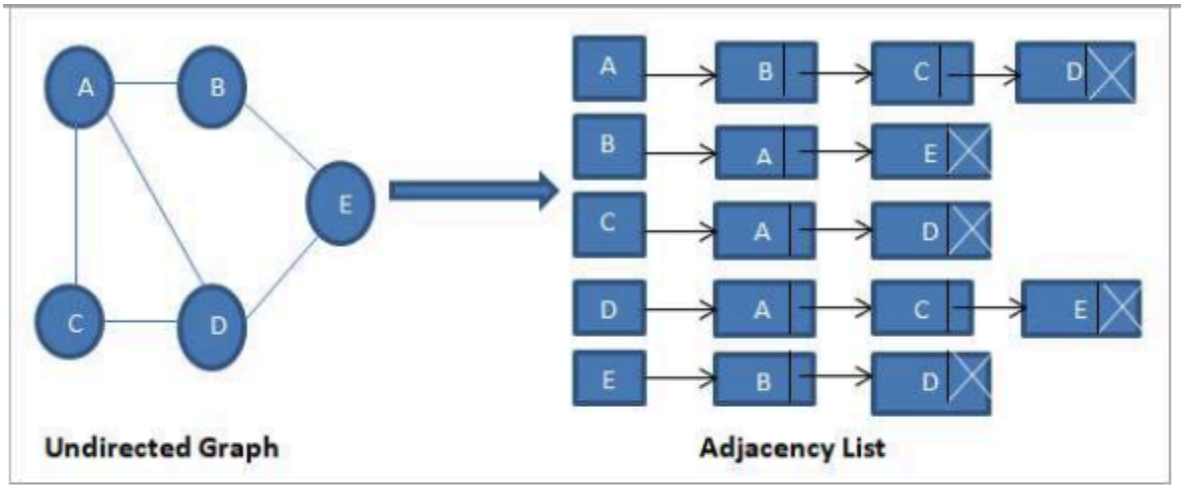


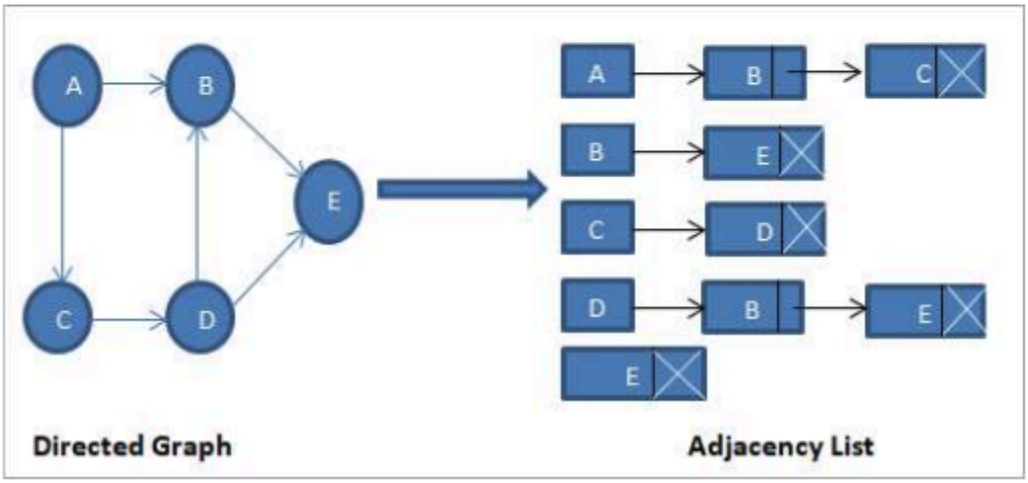
	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	0	1
C	1	0	0	1	0
D	1	0	1	0	1
E	0	1	0	1	0



	A	B	C	D	E
A	0	4	3	0	0
B	0	0	0	0	0
C	0	0	0	2	0
D	0	1	0	0	1
E	1	0	0	0	0

A.3. Adjacency List





Sample Code:

```
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};
// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};
class DiaGraph{
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head)    {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;

        newNode->next = head;   // point new node to current head
     return newNode;
    }
    int N;  // number of nodes in the graph
public:
    adjNode **head;                //adjacency list as array of pointers
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N)  {
        // allocate new node
        head = new adjNode*[N]();
        this->N = N;
        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;
        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++)  {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;
            // insert in the beginning
            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);

            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }
    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++)
            delete[] head[i];
        delete[] head;
    }
};
// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i)
{
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
             << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}
// graph implementation
int main()
{
    // graph edges array.
    graphEdge edges[] = {
            // (x, y, w) -> edge from x to y with weight w
            {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
```

```
    };
    int N = 6;      // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);
    // construct graph
    DiaGraph diagraph(edges, n, N);
    // print adjacency list representation of graph
    std::cout<<"Graph  adjacency  list  "<<std::endl<<"(start_vertex,  end_vertex,
weight):"<<std::endl;
    for (int i = 0; i < N; i++)
    {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }
    return 0;
}
```

Implement the given code and indicate your output as a table in section 6.

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search

Step 1. Include the required header files, as follows:

```
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;
```

Step 2. Write the following struct in order to implement an edge in our graph:

```
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

Step 3. Next, overload the << operator for the graph so that it can be printed out using the following function:

```
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}
```

Step 4. Implement the graph data structure that uses an edge list representation as follows:

```
template <typename T>
class Graph
{
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }
    // Return number of vertices in the graph
    auto vertices() const
    {
        return V;
    }
    // Return all edges in the graph
    auto &edges() const
    {
        return edge_list;
    }

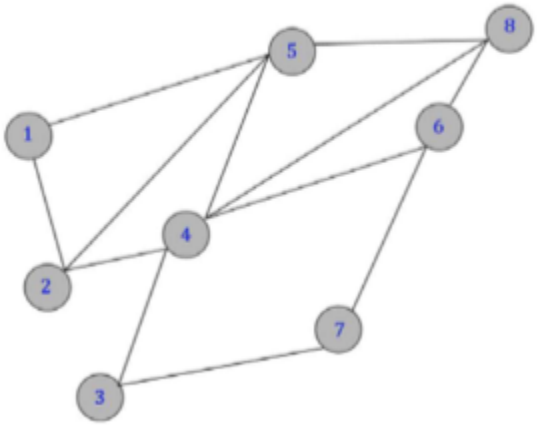
    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }
    // Overloads the << operator so a graph be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
```

```
private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};
```

Step 5. Now, we need a function to perform DFS for our graph. Implement it as follows:

```
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if(visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

Step 6. We shall test our implementation of the DFS on the graph shown here:



Use the following function to create and return the graph:

```
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}
```

Note the use of null values for edge weights since DFS does not require edge weights. A simpler implementation of the graph could have omitted the edge weights entirely without affecting the behavior of our DFS algorithm.

Step 7. Finally, add the following test and driver code, which runs our DFS implementation and prints the output:

```
template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}
int main()
{
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

Step 8. Compile and run the preceding code. **Include your output as a table in section 6.**

B.2. Breadth-First Search

Step 1: Include the required header files and declare the graph as follows:

```
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;
```

Step 2: Write the following struct, which represents an edge in our graph:

```
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

Step 3: Next, overload the << operator for the Graph data type in order to display the contents of the graph:

```
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";

        os << std::endl;
    }
}
```

Step 4: Write a class to define our graph data structure, as shown here:

```
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

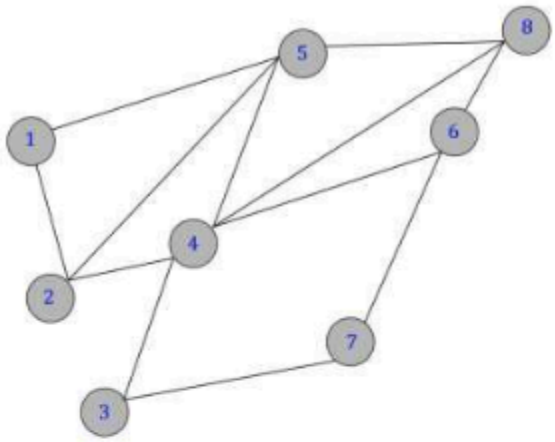
    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    template <typename T>
    friend std::ostream &operator<<(std::ostream &os, const Graph<T> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};
```

Step 5: For this exercise, we shall test our implementation of BFS on the following graph:



We need a function to create and return the required graph. Note that while edge weights are assigned to each edge in the graph, this is not necessary since the BFS algorithm does not need to use edge weights. Implement the function as follows:

```
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}
```

Step 6: Implement the breadth-first search like so:

```
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
```

```
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}
```

Step 7: Add the following test and driver code that creates the reference graph, runs BFS starting from vertex 1, and outputs the results:

```
template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

Include the out and observation as a table in section 6.

ILO A (main.cpp) Code Output:

main.cpp > main()

```cpp
#include <iostream>
// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph{
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head

        return newNode;
    }
    int N; // number of nodes in the graph
public:
    adjNode **head; //adjacency list as array of pointers
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // allocate new node
        head = new adjNode*[N]();
        this->N = N;
        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;
        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;
            // insert in the beginning
            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }
```

```
    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++)
            delete[] head[i];
        delete[] head;
        }
};

// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i)
{
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
            << ", " << ptr->cost << ") ";
    ptr = ptr->next;
    }
    std::cout << std::endl;
}

// graph implementation
int main()
{
    // graph edges array.
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
    };
    int N = 6; // Number of vertices in the graph

    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);

    // construct graph
    DiaGraph diagraph(edges, n, N);

    // print adjacency list representation of graph
    std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex,weight):"<<std::endl;

    for (int i = 0; i < N; i++)
    {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

Terminal Output:

```
Graph adjacency list
(start_vertex, end_vertex,weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

Observations/Analysis:

The code above successfully prints a graph adjacency list using **structures, classes, functions**, and the logical approach that is similar to that of **linked lists** to output a representation of a graph in the format that starts with the **'start_vertex'** followed by the **'end_vertex'** and **'weight'**.

ILO B.1 (DFS.h) Code Output:

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
return os;
}
```

```cpp
template <typename T>
class Graph
{
    public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }
    // Return number of vertices in the graph
    auto vertices() const
    {
        return V;
    }
    // Return all edges in the graph
    auto &edges() const
    {
        return edge_list;
    }
    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }
    // Overloads the << operator so a graph be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};
```

```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if(visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
return G;
}
```

```cpp
template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
    std::cout << v << std::endl;
}
```

Terminal Output:

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

Observations/Analysis:

The code above shows the implementation of a **depth-first search** program and its successful output. It **compares edges, creates multiple helper functions**, and uses the concept of **overloading** to directly write a graph into the stream. Additionally, the search uses the data structure '**stack**' in order to achieve its operation.

ILO B.2 (BFS.h) Code Output:

BFS.h > Graph<T>

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>
#ifndef BFS_H
#define BFS_H



template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";

        os << std::endl;
    }
}
```

```cpp
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
        e.dest >= 1 && e.dest <= V)
        edge_list.emplace_back(e);
    else
        std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
            {
                if (e.src == v)
                {
                    edges_from_v.emplace_back(e);
                }
            }
    return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<T> &G);
```

```
89  private:
90          size_t V;
91          std::vector<Edge<T>> edge_list;
92  };
93
94  template <typename T>
95  auto create_reference_graph()
96  {
97      Graph<T> G(9);
98
99      std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
100     edges[1] = {{2, 2}, {5, 3}};
101     edges[2] = {{1, 2}, {5, 5}, {4, 1}};
102     edges[3] = {{4, 2}, {7, 3}};
103     edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
104     edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
105     edges[6] = {{4, 4}, {7, 4}, {8, 1}};
106     edges[7] = {{3, 3}, {6, 4}};
107     edges[8] = {{4, 5}, {5, 3}, {6, 1}};
108
109         for (auto &i : edges)
110             for (auto &j : i.second)
111                 G.add_edge(Edge<T>{i.first, j.first, j.second});
112     return G;
113 }
```

```cpp
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            queue.push(e.dest);
        }
    }
    return visit_order;
}

template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

#endif
```

Code Error/s:

At the **operator function** at **lines spanning 33-46**, the overloaded output stream operator is said to be **missing the return statement** for **os**. We simply add a '**return os;**' at the very end of the function.

```
In file included from main.cpp:3:
BFS.h: In instantiation of 'std::ostream& operator<<(std::ostream&, const Graph<T>&) [with T = unsigned int; std::ostream = std::basic_ostream<char>]':
BFS.h:143:15:   required from 'void test_BFS() [with T = unsigned int]'
  143 |     std::cout << G << std::endl;
      |     ~~~~~~~~~~~~~^~~~
main.cpp:107:16:   required from here
  107 |     test_BFS<T>();
      |     ~~~~~~~~~~~~^~
BFS.h:46:1: warning: no return statement in function returning non-void [-Wreturn-type]
   46 | }
      | ^
```

Before Fix:	After Fix:

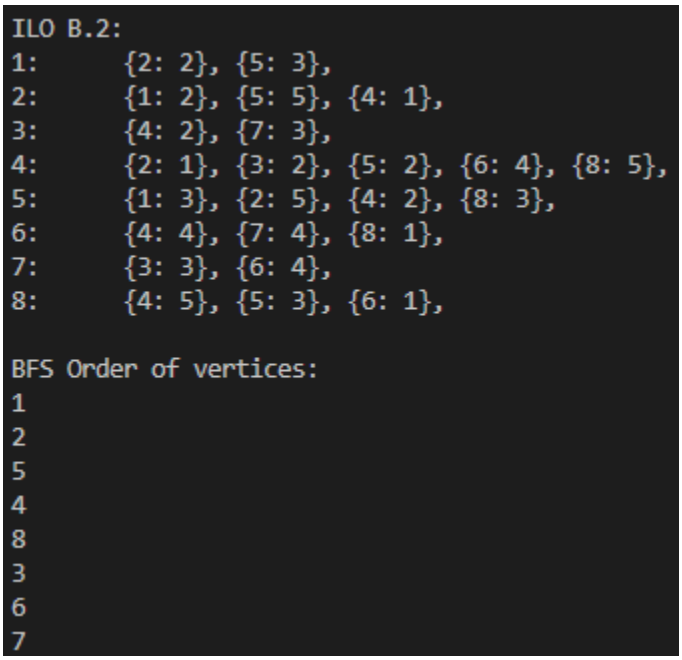
33 template <typename T>
34 std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
35 {
36 for (auto i = 1; i < G.vertices(); i++)
37 {
38 os << i << ":\t";
39
40 auto edges = G.outgoing_edges(i);
41 for (auto &e : edges)
42 os << "{" << e.dest << ": " << e.weight << "}, ";
43
44 os << std::endl;
45 }
46 }
	33 template <typename T>
	34 std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
	35 {
	36 for (auto i = 1; i < G.vertices(); i++)
	37 {
	38 os << i << ":\t";
	39
	40 auto edges = G.outgoing_edges(i);
	41 for (auto &e : edges)
	42 os << "{" << e.dest << ": " << e.weight << "}, ";
	43
	44 os << std::endl;
	45 }
	46 return os;
	47 }

At **line 86**, an error occurs due to the template parameter '**T**' being reused in the same scope, which causes a conflict and makes the program run into an error. To fix this, I simply mirrored what was done to the **DFS.h** library I made and changed this nested template parameter from a '**T**' to a '**U**'.

```
In file included from main.cpp:3:
BFS.h:86:15: error: declaration of template parameter 'T' shadows template parameter
   86 |     template <typename T>
      |               ^~~~~~~~
```

Before Fix:	After Fix:
86 template <typename T>	86 template <typename U>

Terminal Output:



Observations/Analysis:

The code above applies the usage of **breadth-first search** on a graph where, much like with **DFS**, there are **classes, structures, helper functions**, however, instead of **stacks**, the **'queue'** data structure was used. Additionally, **two errors** were found in the code; the overloaded output stream operator is **missing a return statement for 'os'** and a **template parameter was reused within the same scope**. The errors were fixed and the code now runs accordingly.

2. SUPPLEMENTARY OUTPUT

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

   _____
   _____
   _____

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

   _____
   _____
   _____

3. In the performed code, what data structure is used to implement the Breadth First Search?

   _____
   _____
   _____

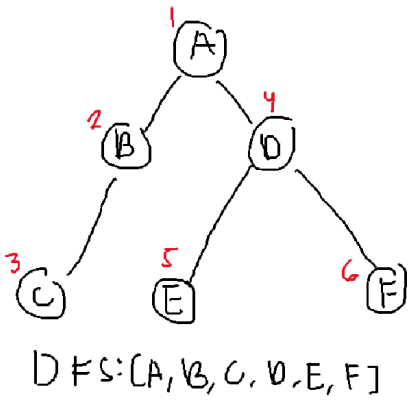
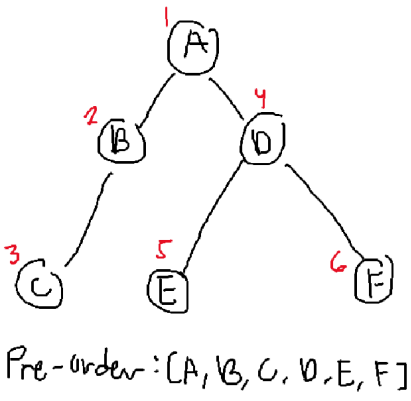
4. How many times can a node be visited in the BFS?

   _____
   _____
   _____

SUPPLEMENTARY ANSWERS:

1. The best algorithm for this task would probably be depth-first search (DFS) as it perfectly matches the logic that the searching algorithm follows. You start from a vertex followed by going through that entire branch before backtracking and exploring another vertex from the same vertex, directly mirroring what our person wants to do. Breadth-first search doesn't really work as it doesn't exactly follow any sort of backtracking, rather, it goes for the oldest discovered nodes first before even moving deeper which doesn't match.

2. The equivalent of DFS in traversal strategies for trees would have pre-, in-, and post-order all within the scope, however, if we want to be real specific, I believe that pre-order is the closest as both follow the principle of finishing a branch before backtracking then heading to the next.

Graphical Comparison	Depth-first Search (DFS)	Pre-order:
		

Pseudocode	```
	DFS(G, u)
	    u.visited = true
	    for each v ∈ G.Adj[u]
	        if v.visited == false
	            DFS(G,v)
	
	init() {
	    For each u ∈ G
	        u.visited = false
	     For each u ∈ G
	       DFS(G, u)
	}
	``
	(Source: Programmiz)
		``
		/* (Based on Programiz)
		function preorderTraversal(node)
		    if node is NULL then
		        return
		    print node.data
		    preorderTraversal(node.left)
		    preorderTraversal(node.right)
		end function
		*/
		``
		(Source: Programmiz)

C++ Code Implementation

```
// DFS algorithm in C++

#include <iostream>
#include <list>
using namespace std;

class Graph {
  int numVertices;
  list<int> *adjLists;
  bool *visited;

   public:
  Graph(int V);
  void addEdge(int src, int dest);
  void DFS(int vertex);
};

// Initialize graph
Graph::Graph(int vertices) {
  numVertices = vertices;
  adjLists = new list<int>[vertices];
  visited = new bool[vertices];
}

// Add edges
void Graph::addEdge(int src, int dest) {
  adjLists[src].push_front(dest);
}
```

(Source: Programmiz)

```
// Tree traversal in C++

#include <iostream>
using namespace std;

struct Node {
  int data;
  struct Node *left, *right;
  Node(int data) {
    this->data = data;
    left = right = NULL;
  }
};

// Preorder traversal
void preorderTraversal(struct Node* node) {
  if (node == NULL)
    return;

  cout << node->data << "->";
  preorderTraversal(node->left);
  preorderTraversal(node->right);
}
```

(Source: Programmiz)

3. In the performed code, the data structure that was used to implement breadth-first search is none other than the 'queue'. Specifically, it uses its functions like push() to add vertices, front() to access adjacent vertices, and pop() to remove them. Since a queue follows the first-in, first-out (FIFO) principle, it should process the vertex that was added the earliest.

4. Nodes are only visited once in BFS as visited vertices are actually recorded. This prevents the nodes from being processed (after they're detected again during the search) and prevents traversal being repeated over and over or even infinitely.

Conclusion:

In this activity, I learned all about depth-first search (DFS) & breadth-first search (BFS), how they work, as well as how they are implemented with a few sessions of bug fixing here and there. I learned how both make use of the stack and queue data structures both in the lab activity and the lecture, as well as how certain coding syntaxes and a missing return statement could break the entire program due to simple typographical mistakes. Additionally, I learned to grasp the more conceptual side of the topic with the supplementary, and even got to compare DFS with the traversal strategies for trees. Overall, this activity has taught me a lot and has set the baseline for what hopefully will appear in our next activities.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.